

test13 多线程 CUDA C++ MultiGrid 测试工作分析报告

opencode analy

2026-08-15

摘要

本报告分析 `logs/test13` 测试套件及其支撑代码在 2026-08-15 的收尾工作：多线程（一线程一卡）`MultiGpuMultigrid` 求解器的正确性验证、性能基准与参数扫描全流程执行，以及测试脚本三处读取 bug 的修复。核心结论：正确性全部 PASS；加速比 `gate=1.5` 通过 11/16 配置，最佳 2.149 (8x8x8x16, L3 r10)；小格子 2L 多线程加速 2.10 倍，中/大格子加速比 < 1 为已知历史特性。三视角概览：主体结构（套件 = 单文件 `main.py` + 运行脚本 + 版本化产物目录）、各部分关系（`main.py` 子命令链 + `h5py` 数据流 + `nullvec` 缓存复用）、项目思路（真多线程并行、加速比语义、数据持久化纯 `h5py`）。

目录

1	仓库概览	1
1.1	分析范围	1
1.2	规模统计与 git 信息	1
2	主体结构	2
3	各部分关系	2
3.1	本次修复的三处读取 bug	2
4	项目思路	2
4.1	设计意图	3
4.2	演进脉络	3
5	主题分析	3
5.1	正确性验证 (verify)	3
5.2	性能基准 (bench)	3
5.3	参数扫描 (sweep) 与 gate	4
6	关键代码片段	4
6.1	粗算子构建统一 C++ <code>matvec</code> 路径	4
6.2	<code>entries</code> 读取统一展开	4
7	参考源清单	4

1 仓库概览

1.1 分析范围

本次分析聚焦 `logs/test13` 套件 (816 行 `main.py` + `run-local.sh` + `AGENTS.md` + 14 项运行产物) 及其核心支撑 `pyqcu/cuda/_multi_gpu.py` (534 行); 主题为本次会话完成的工作: `verify/bench/sweep-/check/collect/mktable/plots/budget` 全流程、`_multi_gpu.py` 粗算子构建统一 C++ `matvec` 路径 (前置未提交改动, 经 `verify` 确认正确)、`main.py` 三处读取 `bug` 修复。

1.2 规模统计与 git 信息

指标	实测值
套件代码文件	<code>main.py</code> (816 行) + <code>run-local.sh</code> + <code>AGENTS.md</code>
运行产物	14 项 (<code>h5/tex/png</code>) 归档于 <code>v202608151626/</code>
<code>nullvec</code> 缓存	8 个粗算子 <code>h5</code> (0.1–2.8 GB)
提交数	最新提交 <code>eef6bbf</code> (2026-08-15 04:31)
最近标签	<code>test12_2</code> (2026-08-14)

表 1: 分析对象实测统计 (数据来源: `wc/git/ls` 实测)

2 主体结构

3 各部分关系

依赖主链: `run-local.sh` → `main.py` 子命令 → `MultiGpuMultigrid` → `CudaSchurOp` ×N 线程 (C++ `libqcu.so` 内核) → `nullvec_cache` 粗算子复用 → `h5` 产物归档版本目录 `v<ts>/`。

要点

数据流: 每线程独立 `params/argv/set_ptrs` (`_SET_INDEX` 各自从 0 计数); 求解热点在 `worker` 线程各自卡上并行 (`qcu.pyx` with `nogil` 真并行); 多线程墙钟 = `max`(各线程时间)。粗算子构建为 `setup` 阶段 (缓存命中秒级), 主线程 `V100` 完成。

3.1 本次修复的三处读取 bug

`collect/mktable/plots/check` 共用 `load_dict_h5` 还原结果, 而 `save_dict_h5` 将 `entries` 列表写为数字 `key` 子组、读取时还原为 `list`; 原读取方按 `dict` 的 `.values()` 处理导致 `AttributeError`。修复为统

层次	路径	职责
入口层	logs/test13/main.py	全部子命令（verify/clean/bench/sweep-/check/ budget/collect/mktable/plots），-outdir 公共参数
编排层	logs/test13/run-local.sh	全流程串联 + timeout 防卡壳 + tee 归档
被测核心	pyqcu/cuda/_multi_gpu.py	MultiGpuMultigrid: N 线程 × 卡并行求解
支撑核心	pyqcu/cuda/_schur_op.py	CudaSchurOp: 每线程独立 C++ matvec 算子
数据层	logs/test13/*.h5 + nullvec_cache/	h5py 结果持久化与粗算子缓存
文档层	logs/test13/AGENTS.md, test13_report.md	复现指南与结果报告

表 2: test13 相关主体结构分层（数据来源: 全库索引实测）

一 `_entries_list()` 展开 (main.py:557)，并新增 `_lat_str()` (main.py:622) 兼容 dataset 还原键 `d_lattice`；budget 子命令的 `--lattices` 解析补 `levels=2` (main.py:806)。

4 项目思路

4.1 设计意图

多线程测试的核心动机：单线程 MG 已收敛 (test12)，需要验证”一线程一卡”在 P100×2 上的真并行加速；基准取多线程 BiStabCG 墙钟而非单线程，衡量的是”并行 MG 相对并行 Krylov 求解器”的加速。数据持久化全部 h5py（独立 File 句柄，多线程安全），避免 json 并发写冲突。

4.2 演进脉络

test11 (dev74 整合) → test12 (单文件优化版) → test13 (多线程版)；本次会话在 test13 上完成收尾：verify 重跑（新代码确认）→ bench 缩小 V100 组格子（16x16x16x32 求解偏慢且缓存不全，超 30min 超时，改为 8x16x16x16）→ sweep/check/collect/mktable/plots/budget 全流程 → 报告。

项目思路

一**句总结**: test13 验证了多线程 CUDA C++ MultiGrid 在小格子上的真并行加速 (2.1 倍)，并以 h5py 全链路持久化保证多线程安全与跨环境可复现。

5 主题分析

5.1 正确性验证 (verify)

全部 PASS: consistency (P100×2, 8x8x8x16, 2L, r5, rel_diff=0.0)、independent (4x4x4x8, 两线程解不同且一致)、V100 单线程 (8x16x16x16, 3L, rel_diff=0.0)、h5py 4 线程 IO 往返零误差。确证——本次会话实测输出 (verify 命令 RC=0, PASS 行 4 条)。

5.2 性能基准 (bench)

配置	nT	ref(s)	mg(s)	speedup
P100x2 8x8x8x16 2L	2	0.571	0.272	2.101
P100x2 8x8x8x16 3L	2	0.575	0.451	1.275
P100x2 8x16x16x16 2L	2	0.613	1.024	0.598
P100x2 8x16x16x16 3L	2	0.598	1.495	0.400
P100x2 16x16x16x16 2L	2	0.695	0.779	0.892
V100 16x16x16x16 3L	1	0.679	0.734	0.926
V100 8x16x16x16 3L	1	0.600	0.528	1.137

表 3: bench 全 7 项 (pairs=3; 数据来源: test13_bench.h5 实测)

小格子 2L 加速显著 (2.10); 3L 反而下降 (粗层迭代开销占比上升); 中格子 < 1 为已知历史特性 (coarse solve 开销, AGENTS.md 「已知硬件特性」)。

5.3 参数扫描 (sweep) 与 gate

16 配置全 PASS; gate=1.5 通过 11/16, 最佳 2.149 (L3 r10 ct1e5 cmi15)。因子均值: 2L=1.874 vs 3L=1.357; r10=1.732 vs r5=1.499; cmi15 优于 cmi10; ct 影响弱。确证——sweep 输出 16 行 OK + check 输出。

6 关键代码片段

6.1 粗算子构建统一 C++ matvec 路径

pyqcu/cuda/_multi_gpu.py:448-461 (未提交改动, 本次 verify 确认正确):

```

1  # 粗算子构建统一走 C++ matvec 路径 (每线程一个 op; 单线程 nthreads=1
2  # 也用 1 个 CudaSchurOp, 避免 Python matvec 构建大格子 50min+ 瓶颈
3  ops_build = [CudaSchurOp(av, g, ce, coo, cei, coi, params=params_t)
4               for _ in range(max(1, min(self.nthreads, 4)))]
5  coarse = build_schur_levels(
6      op, S, self.num_levels, self.dof_list, self.mg_grid, self.lat_size,
7      self.dof_list[1], self.dt, main_dev, nv_iters=self.nv_iters,
8      use_cache=self.use_cache, cache_dir=self.cache_dir, verbose=False,
9      matvec_ops=ops_build, nthreads=len(ops_build),
10     av=av, params_template=params_t)
11  for o in ops_build:

```

12

o.release()

6.2 entries 读取统一展开

logs/test13/main.py:557-574 (本次新增):

1

def _entries_list(d):

2

"""save_dict_h5 的 entries 组还原: {'entries': [{'e0': e}, ...]} →

3

展开为 [e, ...]; 兼容旧 dict 格式 {'e0': e, ...}。"""

4

entries = d['entries']

5

if isinstance(entries, dict):

6

return [entries[k] for k in sorted(entries)]

7

out = []

8

for x in entries:

9

if isinstance(x, dict):

10

out.extend(v for v in x.values())

11

else:

12

out.append(x)

13

return out

7 参考源清单

参考源	类型	内容/作用
logs/test13/main.py:409-429	仓库	_bench_configs: bench 配置表 (V100 组已缩格子)
logs/test13/main.py:557-574	仓库	_entries_list: entries 统一展开 (本次修复)
logs/test13/main.py:622	仓库	_lat_str: 兼容 d_lattice 键 (本次修复)
logs/test13/main.py:806-808	仓库	budget -lattices 解析补 levels=2 (本次修复)
pyqcu/cuda/_multi_gpu.py:448-461	仓库	粗算子构建统一 C++ matvec (未提交)
logs/test13/test13_report.md	仓库	本次收尾的结果报告 (本报告数据源)
logs/test13/AGENTS.md	仓库	套件约定: 参数/加速比语义/硬件特性

8 结论与局限

结论

结论: 结构——test13 为单文件套件 + 版本化产物目录, 与 test12 同构; 关系——main.py 子命令 → MultiGpuMultigrid → CudaSchurOp×N 线程 → nullvec 缓存 → h5 归档, 数据流清晰且多线程安全; 思路——真多线程并行验证达成: 小格子 2L 加速 2.10 倍, gate=1.5 通过 11/16, 最佳 2.149; 正确性全 PASS。

局限与风险

局限: 中/大格子 (8x16x16x16、16x16x16x16) speedup < 1 为已知特性, 本次未做优化尝试 (任务限定验证 + 参数收敛); 16x16x16x32 大格子 bench 未完成 (求解慢 + 粗算子构建超 30min), 已从配置表移除; `_multi_gpu.py` 与 `main.py` 改动未提交 (提示用户自行 git add/commit); 16x16x16x32 残留 2.8GB lv1 缓存 (lv2 缺失不可用) 待清理。